

Individual Practice questions

Here are several scenario-based Python questions that can help software engineering students understand and demonstrate inheritance concepts. They progress from basic to more advanced applications and encourage students to think about real-world software design.

1. University Management System

Scenario

A university stores information about people on campus. Every person has a name and ID number. Students have a major, while lecturers have a department.

Task

1. Create a base class Person.
2. Create subclasses Student and Lecturer that inherit from Person.
3. Add methods to display details of each person.
4. Demonstrate how the inherited attributes are reused.

Learning Objectives

- Basic inheritance
- Constructor inheritance using super()
- Code reuse

2. Vehicle Rental Company

Scenario

A vehicle rental company rents out different types of vehicles. All vehicles have a registration number and rental price per day. Cars have a seating capacity, while motorcycles have an engine capacity.

Task

1. Create a base class Vehicle.
2. Create subclasses Car and Motorcycle.
3. Implement a method to calculate rental cost.
4. Override a method to display vehicle information differently for each subclass.

Learning Objectives

- Inheritance

- Method overriding
- Polymorphism

3. Banking System

Scenario

A bank offers different account types. Every account has an account number and balance. Savings accounts earn interest, while current accounts allow overdrafts.

Task

1. Create a base class BankAccount.
2. Create subclasses SavingsAccount and CurrentAccount.
3. Implement deposit and withdrawal methods.
4. Override withdrawal rules in each subclass.

Learning Objectives

- Inheritance hierarchies
- Method specialization
- Real-world business rules

4. School Examination System

Scenario

A school manages assessments for different types of learners. Every learner has a name and registration number. Undergraduate students are graded out of 100, while postgraduate students are graded using coursework and research scores.

Task

1. Create a parent class Learner.
2. Create child classes Undergraduate and Postgraduate.
3. Implement a method calculate_final_grade().
4. Override the method appropriately in each subclass.

Learning Objectives

- Method overriding

- Domain modeling
- Inheritance design

5. E-Commerce Platform

Scenario

An online store sells physical and digital products. All products have a name and price. Physical products require shipping costs, while digital products require download links.

Task

1. Create a base class `Product`.
2. Create subclasses `PhysicalProduct` and `DigitalProduct`.
3. Implement a method `get_total_cost()`.
4. Show how each product calculates costs differently.

Learning Objectives

- Inheritance and polymorphism
- Extending functionality

6. Employee Payroll System

Scenario

A company pays different categories of employees. Every employee has a name and employee number. Full-time employees earn a monthly salary, while part-time employees are paid hourly.

Task

1. Create a base class `Employee`.
2. Create subclasses `FullTimeEmployee` and `PartTimeEmployee`.
3. Implement a method `calculate_pay()`.
4. Demonstrate different implementations of the same method.

Learning Objectives

- Method overriding
- Polymorphism
- Payroll modeling

7. Online Learning Platform (Multilevel Inheritance)

Scenario

An online learning platform has users. Instructors and students are users. Teaching assistants are also students but have additional teaching responsibilities.

Task

1. Create a class User.
2. Create a class Student inheriting from User.
3. Create a class TeachingAssistant inheriting from Student.
4. Demonstrate access to inherited methods from all levels.

Learning Objectives

- Multilevel inheritance
- Inherited behavior across multiple generations

8. Hospital Information System (Hierarchical Inheritance)

Scenario

A hospital manages different types of staff. Every staff member has a name and staff ID. Doctors have specialties, nurses have wards, and pharmacists have license numbers.

Task

1. Create a parent class Staff.
2. Create subclasses Doctor, Nurse, and Pharmacist.
3. Implement a common method `clock_in()`.
4. Add role-specific methods.

Learning Objectives

- Hierarchical inheritance
- Shared and specialized behavior

9. Smart Home Devices

Scenario

A smart home system controls various devices. All devices can be turned on and off. Smart lights can adjust brightness, while smart thermostats can adjust temperature.

Task

1. Create a base class SmartDevice.
2. Create subclasses SmartLight and SmartThermostat.
3. Reuse power-management methods from the parent class.
4. Add device-specific functionality.

Learning Objectives

- Inheritance for shared functionality
- Extensibility

10. Ride-Hailing Application (Advanced)

Scenario

A ride-hailing company manages different driver types. Every driver has a name, vehicle registration, and rating. Taxi drivers earn commission per trip, while delivery drivers earn commission based on deliveries completed.

Task

1. Create a base class Driver.
2. Create subclasses TaxiDriver and DeliveryDriver.
3. Implement an abstract-like method `calculate_earnings()`.
4. Override the method in each subclass.

Learning Objectives

- Inheritance
- Polymorphism
- Software design principles

Challenge Question (Multiple Inheritance)

Scenario

A university has students who can also be employees. Such individuals need features from both student and employee systems.

Task

1. Create a class Student.
2. Create a class Employee.
3. Create a class StudentEmployee that inherits from both.
4. Demonstrate how attributes and methods from both parents are used.

Learning Objectives

- Multiple inheritance
- Method Resolution Order (MRO)
- Complex object modeling

Question 1: University Management System

Scenario

A university is developing a management system to store information about people associated with the institution.

Every person has:

- Name
- National ID
- Email address

Students additionally have:

- Registration number
- Program of study

Staff members additionally have:

- Employee number
- Department

Some postgraduate students are also teaching assistants. They have both student and staff responsibilities.

Tasks

1. Design a class structure that avoids duplication of common attributes.
2. Implement classes to represent the different categories of people.

3. Create a class for Teaching Assistants that combines both student and staff characteristics.
4. Write methods to display complete information for each person.
5. Demonstrate the use of inherited methods and attributes.

Concepts Demonstrated

- Single inheritance
- Multiple inheritance
- `super()`
- Method Resolution Order (MRO)

Question 2: Smart City Transport System

Scenario

A smart city operates different types of vehicles.

All vehicles have:

- Registration number
- Manufacturer
- Speed

Public buses additionally have:

- Route number
- Passenger capacity

Electric vehicles additionally have:

- Battery percentage
- Charging status

Some buses are electric buses and therefore possess features of both buses and electric vehicles.

Tasks

1. Design an inheritance hierarchy for the transport system.
2. Implement methods to start and stop vehicles.
3. Implement charging functionality for electric vehicles.
4. Create an `ElectricBus` class using multiple inheritance.

5. Demonstrate how inherited methods are accessed.

Concepts Demonstrated

- Hierarchical inheritance
- Multiple inheritance
- Code reuse

Question 3: Hospital Information System

Scenario

A hospital maintains records of employees.

Every employee has:

- Employee ID
- Name
- Contact information

Doctors have:

- Specialization
- Consultation fee

Researchers have:

- Research area
- Number of publications

Some doctors also participate in medical research.

Tasks

1. Create appropriate classes to model the hospital system.
2. Implement methods to display employee details.
3. Create a DoctorResearcher class.
4. Resolve any attribute conflicts that may arise.

Concepts Demonstrated

- Multiple inheritance
- MRO

- Attribute sharing

Question 4: E-Commerce Platform

Scenario

An online shopping platform sells products and offers discounts.

Every product has:

- Product ID
- Name
- Price

A discountable item has:

- Discount percentage
- Discount calculation method

A taxable item has:

- Tax rate
- Tax calculation method

Some products are both taxable and discountable.

Tasks

1. Design the class hierarchy.
2. Implement methods to calculate discounted price.
3. Implement methods to calculate tax.
4. Create a product class that combines both features.
5. Compute the final selling price.

Concepts Demonstrated

- Multiple inheritance
- Reusable business logic
- Method composition

Question 5: Online Learning Platform

Scenario

An online learning platform manages users.

All users have:

- Username
- Email
- Login functionality

Students can:

- Enroll in courses
- View grades

Instructors can:

- Create courses
- Grade assignments

Some users serve as both students and instructors.

Tasks

1. Design a class hierarchy.
2. Implement the required functionalities.
3. Create a class representing users who are both students and instructors.
4. Demonstrate inherited behavior from both parent classes.

Concepts Demonstrated

- Single inheritance
- Multiple inheritance
- Software role modeling

Comprehensive Exam-Style Question

Scenario: Software Engineering Internship Portal

A university wants a portal to manage internship placements.

All users have:

- User ID

- Name
- Email

Students have:

- Registration number
- Course

Company Supervisors have:

- Company name
- Employee ID

Some users are Student Representatives who are both students and supervisors because they coordinate placements while doing internships.

Requirements

1. Create a base class User.
2. Create subclasses:
 - I. Student
 - II. Supervisor
3. Create a class StudentRepresentative that inherits from both Student and Supervisor.
4. Implement methods:
 - I. display_profile()
 - II. login()
 - III. logout()
5. Demonstrate:
 - I. Inherited attributes
 - II. Method overriding
 - III. Multiple inheritance
 - IV. Method Resolution Order using ClassName.mro()